

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4062 PROJE RAPORU

Tiyatro Duyuru Uygulaması

Betül Öğmen

19290263

Danışman : Yılmaz AR

11 Haziran 2024

İÇİNDEKİLER

1.	RAPORUN BÖLÜMLERİ.....	1
1.1.	Özet.....	1
1.2.	Giriş.....	2
1.3.	Gelişme.....	3
1.4.	Sonuç.....	15

1. RAPORUN BÖLÜMLERİ

1.1. Özet

Bu projede, Flutter teknolojisi kullanılarak tiyatro koltuğu seçme uygulaması geliştirilmiştir. Uygulama, kullanıcıların mevcut tiyatroları görüntülemesine, koltuk seçmesine ve bilet rezervasyonu yapmasına olanak tanır. Bu projede, kullanıcı yetkilendirmesi, giriş ekranı, tiyatrolar ekranı, bilet ekranı, koltuk seçme ekranı ve tarihi geçen etkinliklerin yönetimi gibi çeşitli bölümler bulunmaktadır. Uygulamada iki tip kullanıcı rolü vardır: yönetici (admin) ve kullanıcı (user). Yönetici, yeni etkinlikler ekleyebilir, mevcut etkinlikleri silebilir ve tüm etkinlikleri görüntüleyebilirken, kullanıcılar sadece etkinlikleri görüntüleyebilir ve bilet satın alabilir. Bu yapı, uygulamanın güvenliğini sağlamak için önemlidir.

Giriş ekranında, kullanıcılardan e-posta adresi ve şifre talep edilir. Girilen bilgiler Supabase'in kimlik doğrulama API'sine gönderilir ve başarılı bir giriş durumunda oturum tokeni döndürülür. Bu token, uygulamada kullanıcının oturum açık kaldığını gösterir. Ayrıca, uygulamada kayıtlı kullanıcıların bilgileri Supabase'in 'users' tablosunda saklanır. Tiyatrolar ekranında, Supabase veritabanından çekilen etkinlikler listelenir. Etkinlikler 'EventWidget' sınıfı aracılığıyla gösterilir ve her etkinlik için etkinlik adı, türü, yapımcısı, tarihi ve resim içeren bir kart oluşturulur. Yeni etkinlik ekleyebilme yetkisi yalnızca yönetici kullanıcılara tanınır. Etkinliklerin görüntülenmesi ve yönetilmesi, Supabase'in veri yönetimi özelliklerini kullanarak gerçekleştirilir.

Bilet ekranında, tiyatrolar sayfasında seçilen bir tiyatronun üzerine tıklanınca bilet ekranı görünür. Bilet ekranında, tiyatronun adı, yeri, kayıtlı resmi, yapımcısı gibi bilgiler tiyatro ekranından çekilir ve ekranda bilet kodu oluşturulur. 'Koltuk Seç' butonu ile koltuk ekranına gidilirken, 'Bilet Al' ekranı ile bütün değişkenler tamamlanır ve biletin son hali görünür. Koltuk ekranına geçilmeden önce bir pop-up ekranında etkinliğin hangi salondan alınacağı seçilir. Bilet ekranında pop-up ile seçilmiş olan salon numarası doğrulanır ve doğru salondan bilet alınması sağlanır.

Koltuk seçme ekranında tiyatro salonunun dijital versiyonu görünür. Eğer daha önceden alınmış koltuklar var ise, bunlar salonun veri tabanından kontrol edilir. Dolu koltukların rengi ekranda kırmızı, boş koltukların rengi ise yeşil olarak gösterilir. Koltukların numaraları gridlerin üzerinde tanımlıdır ve bu gridlerin üzerine tıklandığında yer ayrılacak koltuklar seçilmiş olur. Tekrar basıldığında koltuklar seçimden kaldırılabilir ve hangi koltuğun seçili olduğu ekranın alt kısmında görünür. 'Yer Ayırt' butonuna basıldığında ise, seçilen koltuk numaraları salonun veri tabanında dolu olarak işaretlenir. Bu sayede o koltuğa o anda başka bir kullanıcı bilet ayıramaz ve koltuk zaten dolu uyarısı alınır.

Bilet Al ekranında, seçilen etkinliğin adı, tarihi, salon numarası, koltuk numaraları ve bilet kodu gösterilir. Bu noktada bilet artık oluşmuştur ve veri tabanındaki 'tickets' tablosunda saklanır.

Proje, her biri belirli bir amaca hizmet eden birkaç Dart dosyasına ayrılmıştır. Bu dosyalar şunlardır: `'ticket_page.dart'` bilet rezervasyon sayfasını yönetir, `'theaters.dart'` mevcut tiyatroları listeler, `'ticket_style.dart'` biletlerin stilini tanımlar, `'tickets.dart'` biletler için veri modelini içerir, `'seat.dart'` koltukların temsili ve işlevselliğini yönetir, `'addseat.dart'` yeni koltukların eklenmesini sağlar ve `'app_color.dart'` uygulamadaki renk temalarını tanımlar.

`Ticket_style.dart` dosyası, uygulamada bilet bilgilerinin görüntülenmesi için kullanılan metin stillerinin tanımlanmasından sorumludur. Bu dosyada, biletlerde olması planlanan değişkenler `theater.dart` dosyasından aktarılır ve bu sayede ilgili verilere erişilir. Sanal bir bilet görüntüsü vermek amacıyla Flutter'ın `TextStyle` sınıfını kullanır ve çeşitli çizgi fonksiyonları ile bilet şekli çizilir. Sayfada biletin öncelikli ve eşsiz değişkeni olan bilet kodu oluşturulur.

`Tickets.dart` dosyası, uygulamadaki bilet konseptini modelleyen bir `Ticket` sınıfını tanımlar. Bu sınıf, 'tickets' tablosuna veri eklenebilmesi için oluşturulmuştur. `Tickets` tablosu, 'ticket_id', 'ticket_type', 'price', 'event_id', 'ticket_date', 'event_name', 'venue', 'event_image', 'user_email', 'ticket_no', 'reserved_seats' ve 'salon' sütunlarından oluşur. 'ticket_id' primary key, 'event_id' ise events tablosu ile bağlantılıdır ve id değerleri events'tan gelir. 'saloonone' ve 'saloontwo' tablolarında birbirine bağlı üç

primary key bulunur: 'event_name', 'seat_no', ve 'event_date'. Bunun böyle tanımlanmasının sebebi, aynı koltuğa iki bilet kesilmesini önlemektir. Bu tablolarda ayrıca koltuğun dolu olup olmadığını tutan bir 'seat_state' sütunu bulunur.

Seat.dart dosyasında, ekranda görünen salon koltukları tasarlanır. Şekillerin düzeni, rengi ve font çeşitleri belirlenir. Koltuklar önce uygulamanın arayüzünde oluşturulur ve sonradan veritabanı işlemleri devreye girer. 'saloonone' ve 'saloontwo' tablolarında daha önceden ayırtılmış koltuk bilgileri tutulur. Bu tablolardan hangi verilerin çekilmesi gerektiği, bilet sayfasından çekilen etkinliğin adı, tarihi ve salonu ile belirlenir. Bu tarihler ile çakışan satırlar var ise, o satırlardaki koltuk numaraları ekranda kırmızıya döner. Ticket_style.dart dosyasında tanımlanmış bilet kodu, sayfa açılır açılmaz oluşturulur.

Giriş ekranında, kullanıcılardan e-posta adresi ve şifre talep edilir. Girilen bilgiler Supabase'in kimlik doğrulama API'sine gönderilir ve başarılı bir giriş durumunda oturum tokeni döndürülür. Bu token, uygulamada kullanıcının oturum açık kaldığını gösterir. Ayrıca, uygulamada kayıtlı kullanıcıların bilgileri Supabase'in 'users' tablosunda saklanır. Tiyatrolar ekranında, Supabase veritabanından çekilen etkinlikler listelenir ve etkinlikler 'EventWidget' sınıfı aracılığıyla gösterilir. Her etkinlik için etkinlik adı, türü, yapımcısı, tarihi ve resim içeren bir kart oluşturulur. Yeni etkinlik ekleyebilme yetkisi yalnızca yönetici kullanıcılara tanınır. Etkinliklerin görüntülenmesi ve yönetilmesi, Supabase'in veri yönetimi özelliklerini kullanarak gerçekleştirilir. Uygulama, sadece tarihi geçmemiş etkinlikleri listeler. Bu, SQL sorgularına eklenen bir koşulla sağlanır ve böylece geçmiş tarihli etkinlikler kullanıcı arayüzünde görünmez. Bu özellik, kullanıcılara güncel ve ilgili bilgilerin sunulmasını sağlar.

Bu metin, projenizin temel işleyişini, kullanılan teknolojileri ve uygulamanın çeşitli ekranlarını detaylı bir şekilde açıklamaktadır. Bu şekilde, raporunuzda hem projenizin genel yapısını hem de teknik detaylarını etkili bir şekilde özetlemiş olursunuz.

1.2. Giriş

Tiyatro gösterilerinin düzenlendiği özel bir tiyatro salonu, etkinliklerini yayımladığı bir online mobile uygulamaya mutlaka ihtiyaç duyacaktır. Etkinliklerin yapılma tarihi, nerede yapılacağını online bir şekilde duyurması gerekecektir. Bu proje öyle bir amaca hizmet etmesi için düzenlenmiştir.

Günümüzde dijital dönüşüm, tüm sektörleri etkilediği gibi sanat dünyasını da etkilemektedir. Tiyatrolar, sanatsal ifadelerini ve etkileşimlerini genişletmek için teknolojiyi benimsemek zorundadır. Bir online mobil uygulama, tiyatroların geniş bir kitleye ulaşmasını sağlayarak sanatın daha erişilebilir olmasına katkıda bulunur. Kullanıcılar, cep telefonları veya tabletler aracılığıyla istedikleri yerden ve zamanda tiyatro etkinlikleri hakkında bilgi sahibi olabilirler. Bu, fiziksel sınırlamaları aşarak sanatseverlerin ve tiyatro meraklılarının tiyatroya olan erişimini genişletir.

Mobil uygulamalar, kullanıcı deneyimini önemli ölçüde iyileştirir. Etkinlik biletlerini online olarak satın almak, koltuk seçimi yapmak, etkinlik programlarını gözden geçirmek ve sanatçılar hakkında bilgi edinmek gibi işlemler kullanıcıların elinin altında olur. Ayrıca, uygulama aracılığıyla kullanıcılara özel indirimler, bildirimler ve güncellemeler sunulabilir, bu da kullanıcıların tiyatro ile olan bağını güçlendirir.

Tiyatrolar için güçlü bir pazarlama ve tanıtım aracıdır. Yeni etkinlikler, özel gösterimler ve sanatçı haberleri hızlı ve etkili bir şekilde duyurulabilir. Push bildirimleri, kullanıcıları yaklaşan etkinlikler ve özel teklifler hakkında bilgilendirir. Bu, etkinliklerin daha fazla insana ulaşmasını ve dolayısıyla daha yüksek bilet satışlarına yol açmasını sağlar.

Kağıt bilet kullanımını azaltarak çevre dostu bir alternatif sunar. Kullanıcılar, uygulama üzerinden biletlerini satın alabilir ve bunları dijital olarak saklayabilirler. Giriş sırasında, biletler QR kodlar veya benzeri dijital yöntemlerle kolayca taranabilir. Bu, hem kullanıcıların hem de tiyatro personelinin işlerini kolaylaştırır ve etkinlik giriş süreçlerini hızlandırır.

Dijital bir platform olarak, mobil uygulamalar, tiyatroların uzun vadeli sürdürülebilirliğini ve gelişimini destekler. Dijital ortamda varlık göstermek, özellikle genç nesiller arasında tiyatroya olan ilgiyi artırabilir ve gelecekteki sanatseverlerin ve destekçilerin yetişmesine katkıda bulunabilir. Ayrıca, dijital platformlar, pandemi gibi olağanüstü durumlarda fiziksel mekanların kapalı olması gibi durumlarda bile sanatsal etkinliklerin devam etmesine olanak tanır. Bu durum, tiyatroların zor zamanlarda bile sanatı sürdürme ve izleyicileriyle bağlarını koruma kapasitesini artırır. Tiyatroların finansal yönetimini ve kaynak optimizasyonunu iyileştirebilir. Online bilet satışları, gelir akışını düzenler ve finansal planlama için değerli veriler sunar. Ayrıca, dijital reklamlar ve sponsorluklar yoluyla ek gelir elde etme fırsatları da sağlar.

Online bir platform, yerel sınırları aşarak küresel bir izleyici kitlesine ulaşmayı mümkün kılar. Bu, uluslararası iş birlikleri ve kültürel alışveriş için yeni kapılar açar. Tiyatrolar, farklı ülkelerdeki sanatçılarla iş birliği yapabilir ve dünya çapında sanatsal projeler geliştirebilirler.

Bu raporun amacı bir online mobil uygulamayı, özel bir tiyatro için yapmaktır.

1.3. Gelişme

Bu projede Flutter kullanarak tiyatro koltuğu seçme uygulaması geliştirildi. Uygulama, kullanıcıların mevcut tiyatroları görüntülemesine, koltuk seçmesine ve bilet rezervasyonu yapmasına olanak tanır.

- Bilet ekranı :

Tiyatro etkinliklerini duyurmak amacıyla yazılan bu uygulamada, yönetici (admin) ve kullanıcı (user) olmak üzere iki tip kullanıcı yetkisi oluşturulmuştur. İki kullanıcının da bilet alma yetkisi vardır.

Tiyatrolar sayfasında seçilen bir tiyatronun üzerine tıklanınca bilet ekranı görünür. Bilet ekranında tiyatronun adı, yeri, kayıtlı resmi, yapımcısı gibi bilgiler tiyatro ekranından çekilir. Ekranda bilet kodu oluşturulur.

'Koltuk Seç' butonu ile koltuk ekranına gidilirken 'Bilet Al' ekranı ile bütün değişkenler tamamlanır ve biletin son hali görünür.

Koltuk ekranına geçilmeden önce bir pop-up ekranında etkinliğin hangi salondan alınacağı seçilir.

- Koltuk ekranı :

Bilet ekranında pop-up ile seçilmiş olan salon numarası doğru salondan bilet alınması için miras alınır. Koltuk seçme ekranında tiyatro salonunun dijital versiyonu görünür. Eğer daha önceden alınmış koltuklar var ise bunlar, salonun veri tabanından kontrol edilir. Dolu koltukların rengi ekranda kırmızı, boş koltukların rengi yeşil renkli görünür. Koltukların numaraları gridlerin üzerinde tanımlıdır. Bu gridlerin üzerine tıklandığında yer ayrılacak koltuklar seçilmiş olur. Tekrar basıldığında koltuklar seçimden kaldırılabilir. Hangi koltuğun seçili olduğu ekranın alt kısmında görünür.

'Yer Ayırt' butonuna basıldığında ise seçilen koltuk numaraları salonun veri tabanında dolu görünür. Bu sayede o koltuğa o anda başka bir kullanıcı bilet ayıramaz. Koltuk zaten dolu uyarısı gelir.

- Bileti Al ekranı:

Bilet Al ekranında seçilen etkinliğin adı, tarihi, salon numarası, koltuk numaraları ve bilet kodu görünür. Bu noktada bilet artık oluşmuştur. Veri tabanındaki 'tickets' tablosunda da bulunur.

Proje, her biri belirli bir amaca hizmet eden birkaç dart dosyasına ayrılmıştır. Ticket_page.dart dosyası bilet rezervasyon sayfasını yönetir. Theaters.dart dosyası mevcut tiyatroları listeler. Ticket_style.dart dosyası biletlerin stilini tanımlar. Tickets.dart dosyası biletler için veri yapısını tanımlar. Seat.dart dosyası koltukların temsili ve işlevselliğini yönetir. App_color.dart dosyası uygulamadaki renk temalarını tanımlar.

Ticket_style.dart dosyası, uygulamada bilet bilgilerinin görüntülenmesi için kullanılan metin stillerinin tanımlanmasından sorumludur. Bu dosyada, biletlerde olması planlanan değişkenler theater.dart dosyasından aktarılır. Bu sayede ilgili verilere

erişilir. Sanal bir bilet görüntüsü vermek amacıyla Flutter'ın TextStyle sınıfını kullanır. Çeşitli çizgi fonksiyonları ile bilet şekli çizilir. Sayfada biletin öncelikli eşsiz değişkeni olacak olan bilet kodu oluşturulur.

```
lib > home > ticket_style.dart > ...
1 import 'dart:math';
2 import 'package:flutter/material.dart';
3 import 'package:theater/home/app_color.dart' as AppColors;
4 import 'package:theater/home/seat.dart';
5 import 'package:theater/screens/ticket_page.dart';
6
7 class TicketStyle extends StatefulWidget {
8   final Map<String, dynamic> eventData;
9   final String saloon;
10  final List<String> selectedSeats;
11
12  const TicketStyle(
13    {Key? key,
14     required this.eventData,
15     required this.saloon,
16     required this.selectedSeats})
17    : super(key: key);
18
19  @override
20  State<TicketStyle> createState() => _TicketStyleState();
21 }
22
23 String ticket_code = generateRandomTicketCode();
24
25 class _TicketStyleState extends State<TicketStyle> {
26   //List<String> reservedSeats = widget.selectedSeats;
27   @override
28   void initState() {
29     super.initState();
30   }
31
32   @override
33   Widget build(BuildContext context) {
34
35     class _TicketStyleState extends State<TicketStyle> {
36       Widget build(BuildContext context) {
37
38         void _showSaloonSelectionDialog() {
39           showDialog(
40             context: context,
41             builder: (BuildContext context) {
42               return AlertDialog(
43                 title: Text('Salon Seçimi'),
44                 content: Column(
45                   mainAxisAlignment: MainAxisAlignment.min,
46                   children: [
47                     ElevatedButton(
48                       onPressed: () {
49                         setState(() {
50                           _selectedSalon = 'SALON 1';
51                         });
52                         Navigator.pop(context);
53                         Navigator.push(
54                           context,
55                           MaterialPageRoute(
56                             builder: (context) => TheaterScreen(
57                               eventData: widget.eventData,
58                               saloon: _selectedSalon,
59                               selectedSeats: widget.selectedSeats), // TheaterScreen
60                           ), // MaterialPageRoute
61                         );
62                       },
63                       style: ElevatedButton.styleFrom(
64                         backgroundColor:
65                           AppColors.subtleText, // Button background color
66                         foregroundColor: AppColors.cream, // Text color
67                       ),
68                       child: Text('Salon 1'),
69                     ), // ElevatedButton
70                     ElevatedButton(
71                       onPressed: () {
72                         setState(() {
73                           _selectedSalon = 'SALON 2';
74                         });
75                       },
76
```

Şekil 1-2 ticket_style.dart dosyası

Tickets.dart dosyası, uygulamadaki bilet konseptini modelleyen bir Ticket sınıfını tanımlar. 'tickets' tablosuna veri eklenebilmesi için oluşturulmuştur. Tickets tablosu 'ticket_id', 'ticket_type', 'price', 'event_id', 'ticket_date', 'tevent_name', 'tvenue', 'tevent_image', 'user_email', 'ticket_no', 'reserved_seats', 'salon' sütunlarından oluşuyor. 'ticket_id' primary key, 'event_id' ise events tablosu ile bağlantılı, id değerleri events'tan gelir. 'saloonone' ve 'saloontwo' tablolarında birbirine bağlı üç primary key bulunur. 'event_name', 'seat_no', 'event_date' sütunları birbirine bağlı. Bunun böyle tanımlanmasının sebebi aynı koltuğa iki bilet kesilmesini önlemek. Saloonone ve saloontwo tablolarında koltuğun dolu olup olmadığını tutan bir 'seat_state' sütunu bulunuyor.

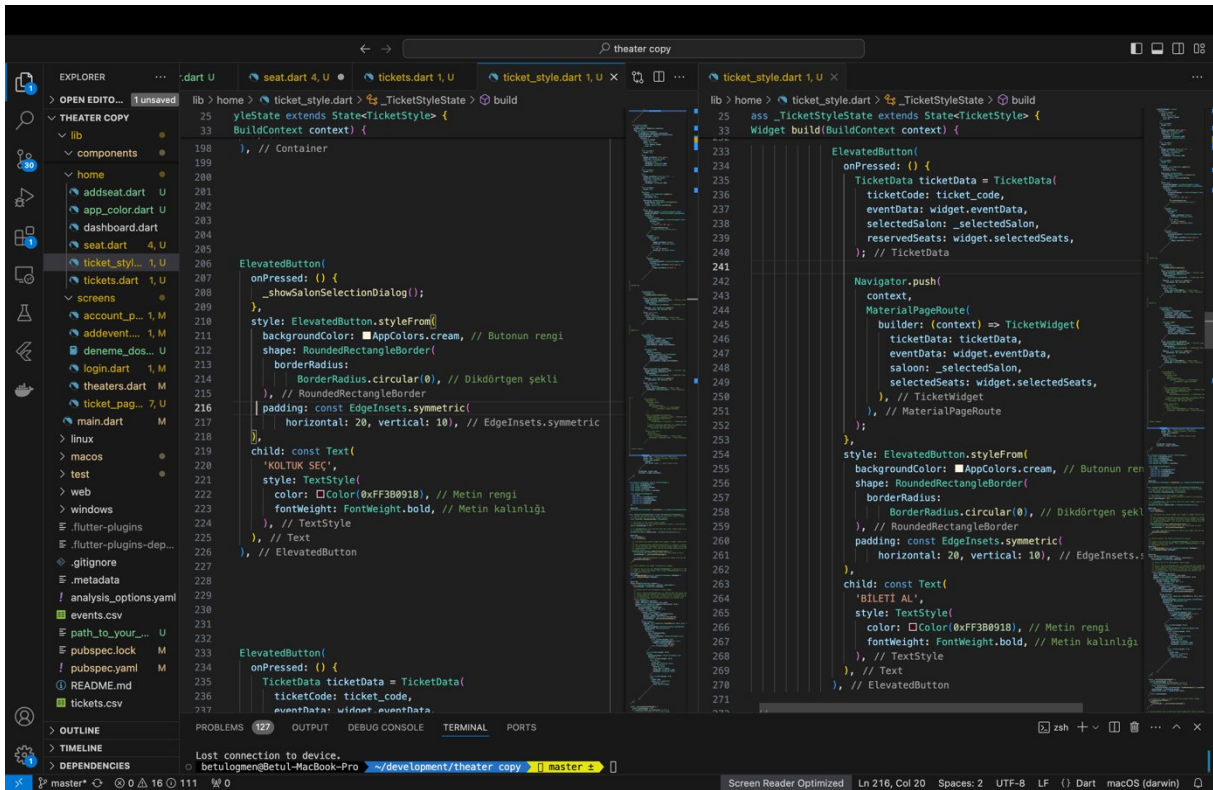
Seat.dart dosyasında ekranda görünen salon koltukları tasarlanır. Şekillerin düzeni, rengi, font çeşitleri belirlenir. Koltuklar önce uygulamanın arayüzünde oluşturulur. Sonradan veritabanı işlemleri devreye girer. 'saloonone' ve 'saloontwo' tablolarında daha önceden ayırılmış koltuk bilgileri tutulur. Bu tablolardan hangi verilerin

çekilmesi gerektiği; bilet sayfasından çekilen etkinliğin adı, tarihi ve salonu ile belli olur. Bu tarihler ile çakışan satırlar var ise o satırlardaki koltuk numaraları ekranda kırmızıya döner. Ticket_style.dart dosyasında tanımlanmış bilet kodu sayfa açılır açılmaz oluşturulur.

```
1 import 'package:flutter/material.dart';
2 import 'package:theater/main.dart';
3 import 'package:theater/screens/login.dart';
4 import 'package:uuid/uuid.dart';
5 import 'package:theater/home/app_color.dart' as AppColors;
6
7 class Ticket extends StatefulWidget {
8   final Map<String, dynamic> eventData;
9
10   const Ticket({Key? key, required this.eventData}) : super(key: key);
11
12   @override
13   _TicketState createState() => _TicketState();
14 }
15
16 class _TicketState extends State<Ticket> {
17   final _formKey = GlobalKey<FormState>();
18   String ticket_id = Uuid().v4();
19   String ticket_type = '';
20   String price = '700';
21   String event_id = Uuid().v4();
22   String user_id = const Uuid().v4();
23   DateTime ticket_date = DateTime.now();
24   late String tevent_name;
25   String tvenue = '';
26   String tevent_image = '';
27
28   bool _hasPermissionToCreateTicket = true;
29
30   @override
31   void initState() {
32     super.initState();
33     _checkPermissionToCreateTicket();
34   }
35 }
```

Şekil 3 ticket.dart dosyası

Ticket_style dosyasında bulunan değişkenler Seat widget'ına aktarılır. Bu biletin ayrılabilmesi için gereklidir. Yer ayrılmak istendiğinde doğru etkinlik bilgileri ile yer ayrılması, TicketStyle widget'ından alınan bilgiler sayesinde mümkün olur.



Ş
e
ki
/
4
K
o
lt
u
k
S
e
Ç
v

e Bilet al butonları

```

1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3 import 'package:theater/home/app_color.dart' as AppColors;
4 import 'package:theater/home/ticket_style.dart';
5
6 class MyApp extends StatelessWidget {
7   final Map<String, dynamic> eventData;
8   final String saloon;
9
10  final List<String> selectedSeats;
11  const MyApp(
12    {Key? key,
13     required this.eventData,
14     required this.saloon,
15     required this.selectedSeats})
16    : super(key: key);
17
18  @override
19  Widget build(BuildContext context) {
20    return MaterialApp(
21      color: AppColors.background, // This uses your custom color
22      title: 'Tiyatro Koltuk Seçimi',
23      theme: ThemeData(
24        primarySwatch: Colors.blue,
25      ), // ThemeData
26      home: TheaterScreen(
27        eventData: eventData,
28        saloon: saloon,
29        selectedSeats: selectedSeats,
30      ), // TheaterScreen
31    ); // MaterialApp
32  }
33
34
35 class TheaterScreen extends StatefulWidget {
36   final Map<String, dynamic> eventData;
37   final String saloon;
38   final List<String> selectedSeats;

```

```

class TheaterScreen extends StatefulWidget {
  final Map<String, dynamic> eventData;
  final String saloon;
  final List<String> selectedSeats;
  const TheaterScreen(
    {Key? key,
     required this.eventData,
     required this.saloon,
     required this.selectedSeats})
    : super(key: key);

  @override
  _TheaterScreenState createState() => _TheaterScreenState();
}

class _TheaterScreenState extends State<TheaterScreen> {
  late List<List<SeatStatus>> theaterLayout;
  List<String> selectedSeats = [];

  String get salon_no => widget.saloon;

  @override
  void initState() {
    super.initState();
    // Initialize the theater layout

    theaterLayout = List.generate(
      10,
      (_) => List.generate(22, (_) => SeatStatus.available),
    ); // List.generate
    _loadOccupiedSeats();
  }

  @override
  Widget build(BuildContext context) {
    // ...

```

Şekil 5-6 seat.dart dosyası

supabase.com/dashboard/project/ttuangxwymejnlvtrws/editor/29572

Table Editor

Nur-betül's Org Free Ticket_Page_DB Enable branching

Filter Sort Insert

RLS disabled role postgres Realtime off API Docs

ticket_id	uuid	ticket_type	int8	price	text	event_id	uuid	ticket_date	date	tevent_name	to
8b3da818-6470-47c4-bc7e-69c5bdf87b7v		2		700		dd20892c-727d-48c5-bbae-8c15a...		2024-05-05		Xmen	
788dac70-7e10-4918-b15c-f0f4d5001d4d		2		700		dd20892c-727d-48c5-bbae-8c15a...		2024-05-05		Xmen	
8017e977-b543-4424-8191-4f1360ccb3c		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	
2f891f65-d358-4267-8b09-e25fd14d28ad		2		700		9ce9a802-abd7-4610-be79-d9042...		2025-10-02		ekinoks	
9fd3bb28-e81e-45cd-bd7d-d4d5b50d21a		2		700		9ce9a802-abd7-4610-be79-d9042...		2025-10-02		ekinoks	
ada058f0-bb70-4c3f-855f-666030acaa6		2		700		9050d141-d36a-40f1-8309-68efb...		2024-12-12		Event 2	
b384f76a-bb4d-465f-96c6-1a8372ef10a		2		700		4e2c0428-757b-4563-9d9e-f5e41...		2024-12-05		Hamlet	
cbba247a-1c0b-4eea-b599-961b90b4a6		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	
e99b2265-ca62-4c06-9f82-3354dd73cet		2		700		4e2c0428-757b-4563-9d9e-f5e41...		2024-12-05		Hamlet	
ad8a330c-afaa-4fac-899d-c2d518ca69fa		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	
dfc4d078-6da6-4ffa-bab2-c0a84dc1fdda		2		700		4e2c0428-757b-4563-9d9e-f5e41...		2024-12-05		Hamlet	
a565213d-8cd9-4d3c-8d26-0db3ee9e314		2		700		4e2c0428-757b-4563-9d9e-f5e41...		2024-12-05		Hamlet	
59047e47-17d0-4fda-b273-7589f4aaf5d8		2		700		9ce9a802-abd7-4610-be79-d9042...		2025-10-02		ekinoks	
6622f966-8986-4ad6-a1f0-7c7f99e26126		2		700		9050d141-d36a-40f1-8309-68efb...		2024-12-12		Event 2	
10881319-443a-46cb-9c52-55cc39d9836		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	
a9df8b1b-c365-41a8-a7fe-fbaa93896dfb		2		700		9050d141-d36a-40f1-8309-68efb...		2024-12-12		Event 2	
6ad17e95-bca5-44bc-86a3-27cd03d3fddc		2		700		13e6b584-f259-411c-b95a-c29f1d2...		2024-11-04		Satranç	
a5c7f54a-30fb-4e61-b318-4928e7e41a0f		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	
828cc7d9-710a-46c8-b016-e88e10c12849		2		700		873cd162-5dff-4ac8-b5ed-4b378...		2024-12-04		Event 3	
9e8a8a5a-5744-a470-b07f-ae7b0000e...		2		700		60b567ab-7e2e-4264-ae4b-48e3...		2025-03-03		Güllerimiz	

Page 1 of 1 100 rows 26 records Refresh Data Definition

Şekil 7 tickets tablosu

Table Editor: schema: public, Ticket_Page_DB

event_name	seat_no	event_date	seat_status
ekinoks	2	2024-05-12	FALSE
ekinoks	5	2024-05-12	FALSE
ekinoks	94	2025-10-02	TRUE
ekinoks	95	2025-10-02	TRUE
Satranç	52	2024-11-04	TRUE
Satranç	53	2024-11-04	TRUE
Satranç	1	2024-11-04	TRUE
Satranç	2	2024-11-04	TRUE
ekinoks	15	2025-10-02	TRUE
ekinoks	16	2025-10-02	TRUE
Satranç	104	2024-11-04	TRUE
Satranç	106	2024-11-04	TRUE
Satranç	105	2024-11-04	TRUE
ekinoks	55	2025-10-02	TRUE
ekinoks	56	2025-10-02	TRUE
ekinoks	57	2025-10-02	TRUE
ekinoks	58	2025-10-02	TRUE
Hamlet	88	2024-12-05	TRUE
Hamlet	89	2024-12-05	TRUE
Hamlet	90	2024-12-05	TRUE

Page 1 of 1, 100 rows, 48 records

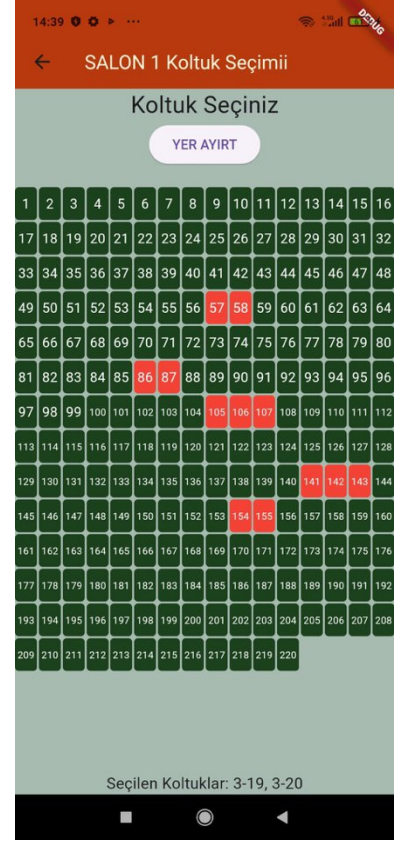
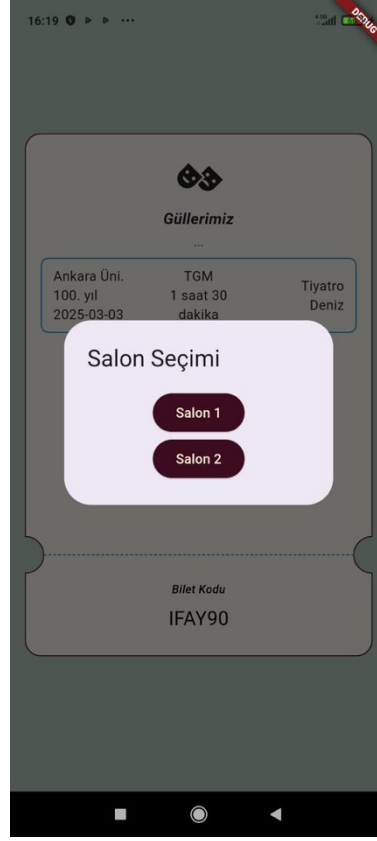
Şekil 8 saloonone tablosu

Table Editor: schema: public, Ticket_Page_DB

event_name	seat_no	event_date	seat_status
Güllerimiz	76	2025-03-03	TRUE
Güllerimiz	77	2025-03-03	TRUE
Güllerimiz	78	2025-03-03	TRUE
Event 2	150	2024-12-12	TRUE
Event 2	151	2024-12-12	TRUE
Event 2	152	2024-12-12	TRUE

Page 1 of 1, 100 rows, 6 records

Şekil 9 saloontwo tablosu



Şekil 10-11-12 Bilet ekranı, Salon seçim ekranı, Koltuk seçim ekranı



Şekil 13 Bilet oluşturma ekranı

Giriş ekranı, kullanıcılardan e-posta adresi ve şifre talep eder. Girilen bilgiler Supabase'in kimlik doğrulama API'sine gönderilir ve başarılı bir giriş durumunda oturum tokeni döndürülür. Bu token, uygulamada kullanıcının oturum açık kaldığını gösterir. Ayrıca, uygulamada kayıtlı kullanıcıların bilgileri Supabase'in 'users' tablosunda saklanır. Tiyatrolar ekranında, Supabase veritabanından çekilen etkinlikler listelenir. Etkinlikler 'EventWidget' sınıfı aracılığıyla gösterilir ve her etkinlik için etkinlik adı, türü, yapımcısı, tarihi ve resim içeren bir kart oluşturulur. Yeni etkinlik ekleyebilme yetkisi yalnızca yönetici kullanıcılara tanınır. Etkinliklerin görüntülenmesi ve yönetilmesi Supabase'in veri yönetimi özelliklerini kullanarak

gerçekleştirilir.

Uygulama sadece tarihi geçmemiş etkinlikleri listeler. Bu, SQL sorgularına eklenen bir koşulla sağlanır, böylece geçmiş tarihli etkinlikler kullanıcı arayüzünde görünmez. Bu özellik, kullanıcılara güncel ve ilgili bilgilerin sunulmasını sağlar.

Tiyatro etkinliklerini duyurmak amacıyla yazılan bu uygulamada, yönetici (admin) ve kullanıcı (user) olmak üzere iki tip kullanıcı yetkisi oluşturulmuştur. İki kullanıcının da bilet alma yetkisi vardır. Tiyatrolar sayfasında seçilen bir tiyatronun üzerine tıklanınca bilet ekranı görünür. Bilet ekranında, tiyatronun adı, yeri, kayıtlı resmi, yapımcısı gibi bilgiler tiyatro ekranından çekilir. Ekranda bilet kodu oluşturulur.

Bilet ekranında pop-up ile seçilmiş olan salon numarası doğru salondan bilet alınması için miras alınır. Koltuk seçme ekranında tiyatro salonunun dijital versiyonu görünür. Eğer daha önceden alınmış koltuklar var ise bunlar salonun veri tabanından kontrol edilir. Dolu koltukların rengi ekranda kırmızı, boş koltukların rengi yeşil renkli görünür. Koltukların numaraları gridlerin üzerinde tanımlıdır. Bu gridlerin üzerine tıklandığında yer ayrılacak koltuklar seçilmiş olur. Tekrar basıldığında koltuklar seçimden kaldırılabilir. Hangi koltuğun seçili olduğu ekranın alt kısmında görünür.

App_colors.dart dosyası uygulamada bilet bilgilerinin görüntülenmesi için kullanılan metin stillerinin tanımlanmasından sorumludur. Bu dosyada, biletlerde olması planlanan değişkenler theater.dart dosyasından aktarılır. Bu sayede ilgili verilere erişilir. Sanal bir bilet görüntüsü vermek amacıyla Flutter'ın TextStyle sınıfını kullanır.

Flutter, Google tarafından geliştirilen açık kaynaklı bir UI yazılım geliştirme kitidir. Mobil, web ve masaüstü uygulamaları geliştirmek için kullanılır. Flutter'ın benimsenmesinin ardındaki en büyük nedenlerden biri, aynı kod tabanını kullanarak çeşitli platformlarda uygulamalar geliştirme yeteneğidir. Bu, geliştirme süresini ve maliyetini önemli ölçüde azaltır.

Hızlı Geliştirme: Flutter'ın "hot reload" özelliği, kod değişikliklerinin anında görülebilmesini sağlar. Bu, geliştirme sürecini hızlandırır ve geliştiricilerin hızlıca iterasyon yapmasına olanak tanır.

Yerel Performans:Flutter, doğrudan makine diline derlenir ve bu da yüksek performanslı uygulamalar oluşturulmasına olanak tanır.

Zengin Widget Kütüphanesi: Flutter, önceden oluşturulmuş geniş bir widget kütüphanesi sunar. Bu widget'lar, platformdan bağımsız olarak çalışır ve her iki platformda da (iOS ve Android) yerel bir görünüm ve his sunar.

UI Tasarımı: Flutter, uygulamanızın kullanıcı arayüzünü (UI) oluşturmak için bir dizi özelleştirilebilir widget sunar. Bu widget'lar, yerleşim, stil ve animasyonları kolayca yönetmenizi sağlar.

State Management: Flutter'da durumu yönetmek için çeşitli yaklaşımlar bulunur. En yaygın kullanılan yöntemler arasında Provider, Riverpod ve Bloc yer alır.

Supabase, mevcut uygulamalarınıza gerçek zamanlı veritabanı, kimlik doğrulama ve depolama eklemek için kullanılan açık kaynaklı bir arka uç platformudur. Firebase'e açık kaynaklı bir alternatiftir ve Postgres veritabanı üzerine inşa edilmiştir.

Gerçek Zamanlı Veritabanı: Supabase, veritabanınızda yapılan değişiklikleri gerçek zamanlı olarak uygulamalarınıza iletebilir.

Kimlik Doğrulama: Kullanıcı kayıtları, giriş işlemleri ve üçüncü taraf kimlik doğrulama sağlayıcıları (Google, Facebook, GitHub vb.) ile entegre çalışır.

Depolama: Dosya depolama ve yönetim hizmeti sunar, böylece kullanıcı dosyalarını güvenli bir şekilde saklayabilirsiniz.

Veritabanı Yönetimi: Supabase, PostgreSQL'i temel alır ve bu da güçlü ve ölçeklenebilir bir veritabanı çözümü sunar. SQL sorguları ile veri işleme, saklama ve yönetim işlemlerini kolayca gerçekleştirebilirsiniz.

Kimlik Doğrulama: Supabase Auth, kullanıcıların güvenli bir şekilde kimlik doğrulamasını sağlar ve oturum yönetimini kolaylaştırır.

Depolama: Supabase Storage, dosyaları güvenli bir şekilde yüklemenizi ve yönetmenizi sağlar. Uygulamanızdan medya dosyalarına kolayca erişebilir ve bunları kullanıcılarınıza sunabilirsiniz.

Dart, Google tarafından geliştirilen açık kaynaklı bir programlama dilidir. Flutter uygulamaları geliştirmek için kullanılan birincil dildir. Dart, hem istemci tarafında (UI) hem de sunucu tarafında (arka uç) çalışabilir.

Basit ve Hızlı: Dart, öğrenmesi kolay bir sözdizimine sahiptir ve hızlı derleme süreleri sunar.

Objektif Yönelimli: Dart, nesne yönelimli programlama özelliklerini destekler, bu da büyük ve karmaşık uygulamaların yönetimini kolaylaştırır.

Asenkron Programlama: Dart, asenkron işlemleri yönetmek için güçlü bir desteğe sahiptir. Bu, uygulamanızın performansını ve kullanıcı deneyimini artırır.

Flutter ile Entegrasyon: Dart, Flutter ile sorunsuz bir şekilde çalışır ve Flutter SDK'nın tüm gücünden yararlanmanıza olanak tanır.

Web ve Mobil Geliştirme: Dart, hem web hem de mobil uygulamalar geliştirmek için kullanılabilir, bu da geliştiricilere platformlar arası uygulamalar oluşturma esnekliği sağlar.

Projenin kullanıcı arayüzü tamamen Flutter kullanılarak geliştirildi. Flutter'ın sağladığı hızlı geliştirme süreci ve yerel performans avantajları, projeyi kullanıcı dostu ve performanslı hale getirdi. Dart dili ise, Flutter ile uyumlu çalışarak projenin temel programlama dili olarak kullanıldı. Bu ikili, projenin temel taşlarını oluşturdu.

Projenin arka uç hizmetleri için Supabase kullanıldı. Gerçek zamanlı veritabanı ve kimlik doğrulama özellikleri, uygulamanın güvenli ve hızlı çalışmasını sağladı. Supabase'in sağladığı PostgreSQL tabanlı veritabanı, güçlü ve esnek veri yönetimi çözümleri sundu.

Kalıtım, nesne yönelimli programlamanın (OOP) temel taşlarından biridir. Kalıtım, bir sınıfın başka bir sınıfın özelliklerini ve metotlarını devralmasına olanak tanır. Bu, kod tekrarını azaltır ve daha esnek ve yeniden kullanılabilir yazılım bileşenleri oluşturmayı sağlar. Kalıtım, yazılım mühendisliğinde daha organize ve modüler bir kod yapısı elde etmek için kullanılır.

Bu projede, çeşitli sınıflar arasında kalıtımı kullanarak özellikleri ve işlevsellikleri paylaşmak için kalıtım teknolojisinden yararlanıldı.

`ticket\_style.dart` dosyasında tanımlanan `TicketStyle` sınıfı, biletlerin nasıl görüneceğini tanımlar. `tickets.dart` dosyasında tanımlanan `Ticket` sınıfı ise, bilet bilgilerinin modelini içerir ve `TicketStyle` sınıfının stil özelliklerini kullanır. `TicketStyle` sınıfı, biletlerin metin stillerini tanımlar. Bu sınıf, bilet numarası ve koltuk durumu gibi metinlerin nasıl görüneceğini belirler.

`Ticket` sınıfı, bilet modelini tanımlar ve `TicketStyle` sınıfının özelliklerini kullanarak biletin görünümünü belirler.

`Ticket` sınıfı, `Seat` sınıfının bir örneğini içerir ve biletin fiyatını belirler. `TicketStyle` sınıfı ise, biletin kullanıcı arayüzünde nasıl görüneceğini tanımlar. Bu iki sınıf arasındaki ilişki, biletlerin hem veri modelini hem de stilini belirler ve böylece daha organize bir yapı sağlar.

Projedeki sayfalar arasında veri aktarımı, kalıtım ve Flutter'ın widget yapısı kullanılarak gerçekleştirilir. `ticket\_page.dart`, `theaters.dart` ve `seat.dart` dosyaları, sayfalar arasında veri aktarımını ve kullanıcı etkileşimini sağlar.

`TicketPage` sınıfı, `Ticket` nesnesini parametre olarak alır ve bu bilgiyi ekranda görüntüler.

`TheatersPage` sınıfı, mevcut biletleri listeleyen bir sayfayı tanımlar. Kullanıcı bir bileti seçtiğinde, `TicketPage` sayfasına yönlendirilir ve seçilen biletin detayları gösterilir.

`TicketPage` sınıfı, `TheatersPage` sınıfından seçilen bilet verisini alır ve bu veriyi kullanıcıya gösterir. Bu, sayfalar arasında veri aktarımını ve kullanıcı etkileşimini sağlar.

Kalıtım, projenin daha modüler ve yönetilebilir olmasını sağlar. Kod tekrarını azaltır ve sınıflar arasında ortak işlevlerin paylaşılmasını kolaylaştırır. Ayrıca, sınıflar arasında açık bir hiyerarşi oluşturarak kodun anlaşılmasını ve bakımını kolaylaştırır.

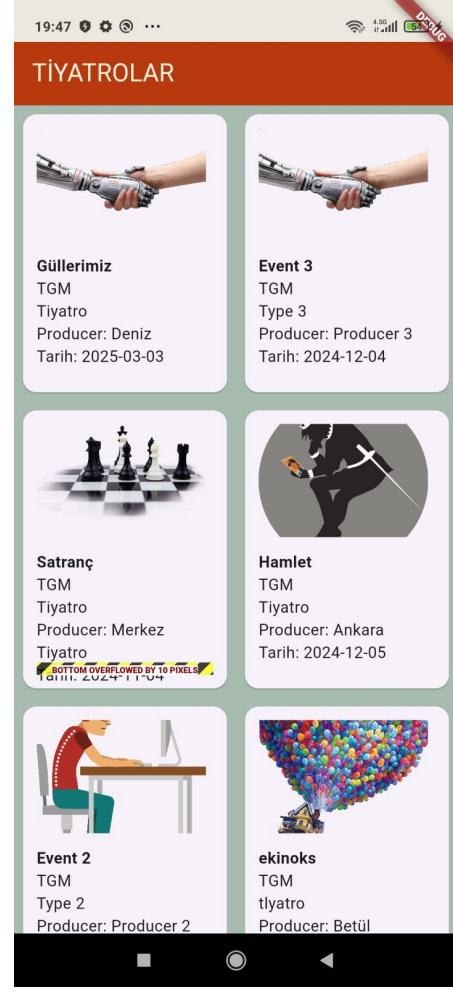
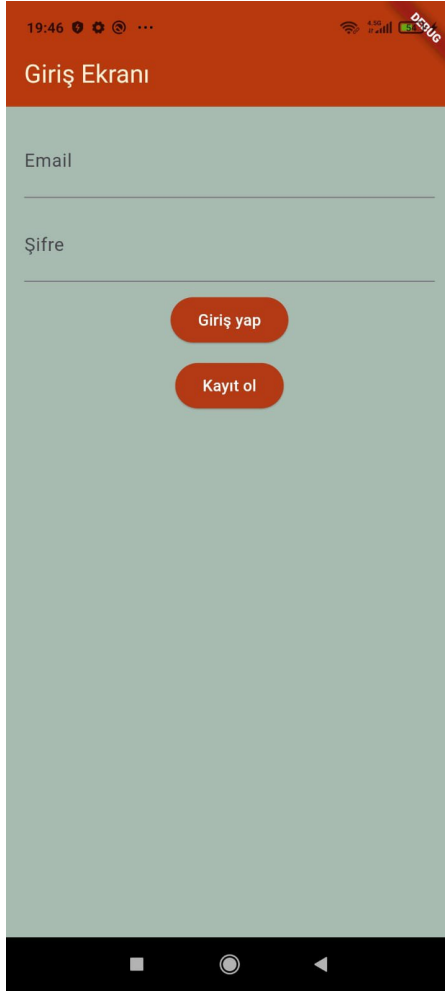
Kod Tekrarını Azaltma: Kalıtım, aynı kodun birden fazla yerde yazılmasını önler.

Modülerlik: Sınıflar arasında ortak işlevler paylaşılabilir, bu da kodun modüler olmasını sağlar.

Bakım Kolaylığı: Kalıtım, kodun anlaşılmasını ve bakımını kolaylaştırır, çünkü kod yapısı daha düzenli ve hiyerarşik olur.

Ticket ve TicketStyle sınıflarında bilet bilgilerini ve stillerini yönetmek için kalıtım kullanıldı. Kullanıcı etkileşimini ve veri aktarımını sağlamak için kalıtım ve Flutter widget yapısı kullanıldı.

Bu detaylı açıklamalar, projede kullanılan kalıtım teknolojilerini ve bunların nasıl uygulandığını kapsamlı bir şekilde açıklar. Raporunuzda bu metinleri kullanarak projenizin teknolojik altyapısını vurgulayabilirsiniz.



Şekil 10-11 Giriş ekranı ve Tiyatrolar ekranı

https://github.com/Nur-betul/theater_app.git

1.4. Sonuç

Bu rapor, bir tiyatro salonunun etkinliklerini online mobil uygulama aracılığıyla duyurmasının önemini ve etkinliğini ele almaktadır. Projede geliştirilen mobil uygulama, tiyatroların dijital dönüşümüne katkı sağlamak ve geniş bir kitleye ulaşmasını kolaylaştırmak amacıyla tasarlanmıştır. Uygulama, kullanıcıların tiyatro etkinlikleri hakkında kolayca bilgi edinmelerine, bilet satın almalarına ve etkinlik programlarına erişmelerine olanak tanır.

Projede geliştirilen mobil uygulama, tiyatroların dijital dünyada varlık göstermesini sağlayabilir ve genç nesiller arasında tiyatroya olan ilgiyi artırabilir. Dijital biletler ve

kolay erişim özellikleri ile hem kullanıcı deneyimi iyileştirilmiş hem de çevre dostu bir alternatif sunulmuştur.

Netice olarak, bu tarz bir uygulama, tiyatroların dijital dönüşümüne önemli bir katkı sağlayabilir ve sanatsal etkinliklerin daha geniş bir kitleye ulaşmasını mümkün kılar.

Tiyatro salonlarının, teknolojiyi benimseyerek sanatsal ifadelerini ve etkileşimlerini genişletmeleri, dijital çağda sanatın sürdürülebilirliği ve geniş kitlelere ulaşımı açısından hayati önem taşımaktadır.

Sonuç olarak, bu proje, mobil teknolojinin tiyatroların erişilebilirliğini ve etkileşimini nasıl artırabileceğinin somut bir örneğini sunmaktadır. Projede geliştirilen uygulama, hem tiyatrolar için hem de sanatseverler için değerli bir kaynak oluşturmuştur. Bu başarı, dijitalleşmenin tiyatrolar için sadece bir seçenek değil, aynı zamanda gelecekteki başarının anahtarı olduğunu kanıtlamaktadır.

2. KAYNAKÇA

<https://supabase.com/docs/guides/getting-started/quickstarts/flutter>

<https://supabase.com/docs/guides/getting-started/tutorials/with-flutter>

<https://supabase.com/docs/reference/dart/insert>

<https://supabase.com/docs>